

COP3330

University of Central Florida

Linked list study notes

This file contains the 3 classes used in the linked list implementations in the COP3330 class. Please write the methods by yourself then check the code implementations here so that you can develop yourself.

```
package lectureLinkedList;
/*
 * Node class in Linked List
 * COP3330
 * Author: Safa
 */
public class Node {

    private int item;
    private Node next;

    // we dont need any default node in our
    implementations
    // so we are stopping the developers calling
    no arg constructor
    private Node() {
        next=null;
        item=0;
    }

    public Node(int gitem) {
        item=gitem;
        next=null;
    }
}
```

```
    }

    public Node getNext() {
        return next;
    }

    public int getItem() {
        return item;
    }

    public void setNext(Node n) {
        next=n;
    }

    //please do not use this method
    //in your implementations although as an
    //ethical rule this method is
    //implemented also.
    private void setItem(int gitem) {
        item=gitem;
    }
}
```

```
package lecturelinkedList;
/*
 * Linked List without dummy head node
 * COP3330
 * Author: Safa
 */
public class LinkedList {
    private Node head;

    public LinkedList(){
```

```
        head=null;
    }

    //copy constructor making deep copy
    public LinkedList(LinkedList tobecopied) {
        head=null;
        Node cur=tobecopied.getHeadNode();
        Node tailOfMyList=null;
        while(cur!=null) {
            Node n = new Node(cur.getItem());
            //if tailOfMyList is null that means
we need to add to head
            // it means this is the first
iteration
            if(tailOfMyList ==null) {
                tailOfMyList=n;
                head=n;
                //printing for debugging purposes

                System.out.println(cur.getItem()+" is added
to head");
            }else {
                tailOfMyList.setNext(n);

                tailOfMyList=tailOfMyList.getNext();
                //we need to update the
tailOfMyList so that we can add the
                //other nodes in the next
iterations
                // tailOfMyList always points to
the tail node
                //printing for debugging purposes

                System.out.println(cur.getItem()+" is added
to tail");
            }
        }
    }
}
```

```

        }
        cur=cur.getNext();
    }
}

public boolean isEmpty() {
    return null == head;
}

//this method is needed so that we can
//call copy constructor
public Node getHeadNode() {
    return head;
}

public void printAll() {
    if(!isEmpty()) {
        Node cur=head;
        while(cur!=null) {
            System.out.print(cur.getItem()+"
");
            cur=cur.getNext();
        }//end of while
        System.out.println();
    }else {
        System.out.println("No element
inside");
    }
}

public void addHead(int y) {
    Node n = new Node(y);
    n.setNext(head);
    //
    //      {n      }  ->      { head}

```

```
        //
        head=n;
    }

    public void addTail(int y) {
        Node n =new Node(y);
        if(isEmpty()) {
            head=n;
        }else {
            Node cur=head;
            while(cur.getNext()!= null) {
                cur=cur.getNext();
            }
            //right now we are at tail
            cur.setNext(n);    // cur.next=n
        }
    }

    public int removeHead() {
        int returned=-1;
        if(!isEmpty()) {
            Node cur=head; //we dont need this
            returned=head.getItem();
            head=head.getNext();
            cur.setNext(null);
            // cur is added here to show that in
            // C++ implementation we will need to
            // free it as : free(cur)
        }
        return returned;
    }

    public int removeTail() {
        int returned=-1;
        if(!isEmpty()) {
```

```
        if(head.getNext()==null) {
            //there is only one element
            returned=head.getItem();
            head=null;
        }else {
            //Node cur=head;
            Node cur=head;

            for(;cur.getNext().getNext()!=null;cur=cur.g
etNext()) {
                //we dont need to do anything
                here
            }//end of for
            //right now I am right
            behind/before the tail
            returned=cur.getNext().getItem();
            cur.setNext(null);
        }
    }//end of if empty check
    return returned;
}

public void fill(int[] arr) {
    for(int el:arr) {
        addTail(el);
    }
}

public static void p(String x) {
    System.out.println(x);
}

public void remove(int y) {
    if(!isEmpty()) {
        if(head.getItem()==y) {
```

```
        head=head.getNext();
    }else {
        Node cur=head;
        while(cur.getNext()!=null &&
cur.getNext().getItem()!=y) {
            cur=cur.getNext();
        }//end of while
        if(cur.getNext()==null) {
            p("No element");
        }else {

cur.setNext(cur.getNext().getNext());
        }
        }//end of if

    }else {
        p("Empty list nothing to remove");
    }

}

//adding el right after the y element
public void addAfter(int y, int el) {
    Node cur=head;
    while(cur!=null && cur.getItem()!=y) {
        cur=cur.getNext();
    }
    //if the linked list empty
    // if cur is null it means we couldnt
find it
    if(cur != null) {
        Node n =new Node(el);
        n.setNext(cur.getNext());
        cur.setNext(n);
    }else {
```

```
        p("Linked list empty or no "+y+"
found");
    }
}

public static void main(String[] args) {
    LinkedList ll =new LinkedList();
    int[] arr= {1,2,3,4,5};
    ll.fill(arr);
    ll.printAll();
    ll.addTail(6);
    ll.printAll();

    ll.remove(6);
    p("All the list");
    ll.printAll();

    ll.addAfter(3,-1);
    p("All the list");
    ll.printAll();

    LinkedList mycopy=new LinkedList(ll);
    //calling copy constructor
    p("printing the copy list");
    mycopy.printAll();

    ll.remove(2);
    p("printing the copy list after removal
from original");
    mycopy.printAll();

}
}
```

```
package lectureLinkedList;
/*
 * Linked List with dummy head node
 * COP3330
 * Author: Safa
 */
public class DHLinkedList {

    Node head;

    public DHLinkedList() {
        Node dummy=new Node(0);
        head=dummy;
    }

    public boolean isEmpty() {
        return null == head.getNext();
    }

    public void addHead(int y) {
        Node n =new Node(y);
        n.setNext(head.getNext());
        head.setNext(n);
    }

    public int removeHead() {
        int returned=-1;
        if (!isEmpty()) {
            returned=head.getNext().getItem();

            head.setNext(head.getNext().getNext());
        }
    }
}
```

```

        return returned;
    }

    // there is always a node before y
    //although the list is empty
    // compare this method with the linked list
    without dummy head node
    public void remove(int y) {
        Node cur=head;
        while( cur.getNext() !=null &&
cur.getNext().getItem()!=y) {
            cur=cur.getNext();
        }
        if(cur.getNext()==null) {
            LinkedList.p("No such element");
        }else {
            cur.setNext(cur.getNext().getNext());
        }
    }

    //add el right after y
    public void add(int y, int el) {
        Node n =new Node(el);
        Node cur=head.getNext();
        while( cur !=null && cur.getItem()!=y) {
            cur=cur.getNext();
        }
        if(cur==null) {
            System.out.println("No such item or
list is empty");
        }else {
            n.setNext(cur.getNext());
            cur.setNext(n);
        }
    }

```

```
    }

    public void printAll() {
        Node cur=head.getNext();
        while(cur!=null) {
            System.out.print(cur.getItem()+" ");
            cur=cur.getNext();
        }
        System.out.println(" ");
    }

    public static void main(String[] args) {
        DHLinkedList ll =new DHLinkedList();

        ll.printAll();
        ll.addHead(1);
        ll.addHead(2);
        ll.addHead(3);
        ll.addHead(4);

        ll.printAll();

        ll.add(2, -1);
        System.out.println("Printing list After
adding -1 right after 2");
        ll.printAll();
    }
}
```

Notes:

Please compare the method implementations of linked list with dummy node and the linked list without dummy node. Adding dummy node saves us checking if list is empty in our implementation in remove. Adding a dummy head node makes the code implementation easier.

As an exercise you can write copy constructor in the dummy head node linked list.